Керування 74HC595: від теорії до схеми в KiCad

Урок 4 — 2МАК, SST-MEC-25c — Регістр зсуву на практиці

01/10/2026

1 Повторення: що таке 74HC595

Примітка

D-тригер — компонент із двома входами (**CLK**, **D**) і двома виходами (Q, $\overline{Q}$). Щоразу, коли провід **CLK** отримує **передній фронт** (перехід з низького рівня на високий), тригер "**робить фотографію**" рівня, який у цю мить є на проводі D, і зберігає його на Q: провід D може вільно змінюватися з часом, але Q **залишається незмінним** на останньому сфотографованому значенні, доки не прийде новий передній фронт. Отже, це **комірка пам'яті**.

Примітка

Асинхронний лічильник, який ми вже бачили, — це **батарея D-тригерів**, з'єднаних каскадом: він починає з **усіх нулів** і на кожному передньому фронті **збільшує на одиницю** двійкове число, яке показують виходи. З 4 тригерами можна дійти до 1111 (15 у десятковій), а потім лічба **повертається до нуля** – щоб показувати більші числа, потрібно **більше тригерів** у каскаді. Він **асинхронний**, бо clock не надходить прямо до всіх тригерів одночасно: лише перший отримує "справжній" **clock**, а кожним наступним ступенем керує **вихід попереднього ступеня**.

Примітка

Кожен наступний ступінь лічильника **удвічі зменшує частоту змін** порівняно з попереднім: перший біт змінюється на кожному фронті clock, другий — **кожні два фронти**, третій — **кожні чотири** і так далі. Це зменшення вдвічі пов'язане зі співвідношенням між **частотою і періодом**:

$$f = \frac{1}{T}$$

Якщо **період** (час між однією подією та наступною) подвоюється, то **частота** (кількість подій за секунду) **зменшується вдвічі** – і саме це відбувається **від ступеня до ступеня** в асинхронному лічильнику.

Визначення

74HC595 ("**595** для друзів") — це **8-бітний** послідовно-паралельний **регістр зсуву** (*shift register*). "Послідовно-паралельний" означає, що дані **входять** по одному (**послідовно**, біт за бітом, як кульки, які по одній вкладають у трубку), але їх можна **прочитати всі разом**

паралельно на 8 вихідних виводах.

Примітка

Позначення компонента читають так: **74** вказує на **сімейство** (**комерційне** **застосування**, на відміну від сімейства 54 для військового застосування, з ширшим діапазоном температур); **дві** **літери** посередині (HC, LS, HCT, ...) вказують на **технологію** **виготовлення** внутрішньої схеми – це не інший фізичний матеріал, а інший спосіб усередині реалізувати ту саму логічну функцію. **HC** (High-speed CMOS) — найсучасніша технологія з **малим** **споживанням**, сьогодні майже завжди доступна; **LS** — старіша технологія, швидша, але з **більшим** **споживанням**. На електричній схемі важить лише **загальний** **символ** 74HC595: якщо потім фізично вдасться дістати 74LS595, це теж підійде, бо **логічна** **функція** однакова – змінюється лише **технологія** **виготовлення**, деталь, яка стосується тих, хто проєктує складні пристрої, а не рівня цього курсу.

Приклад

Аналогія з **3D-друком**: той самий принтер може надрукувати **абсолютно** **той** **самий** **предмет** з **різних** **матеріалів** – **PLA**, **PETG**, **TPU**, **ASA**. Предмет той самий (та сама форма, та сама функція “на папері”), але матеріал змінює його **фізичні** **властивості**: молоток, надрукований з PETG, якщо чимось ударити, **витримає** **удар**; той самий молоток з PLA може **зламатися**, бо PLA **крихкіший**. Так само 74HC595 і 74LS595 “на папері” виконують ту саму логічну функцію, але дві технології виготовлення мають різні **електричні** **характеристики**: HC **споживає** **дуже** **мало**, LS **швидший**, але **споживає** **більше**. Для того, хто проєктує складний пристрій, ця різниця може бути вирішальною у виборі компонента – а для того, хто ще вчиться основ електроніки, як на цьому курсі, це деталь, якій **не** **треба** **надавати** **ваги**: просто беруть той **варіант**, який найлегше знайти в каталозі.

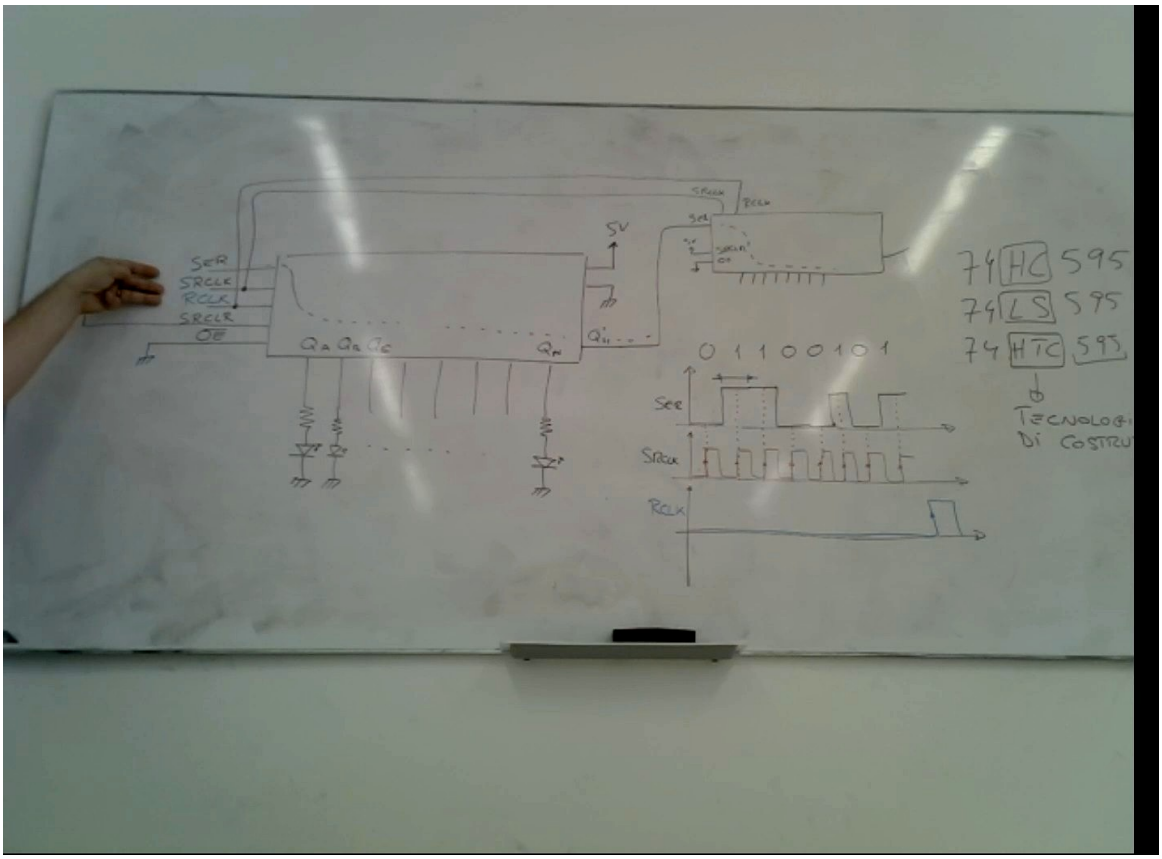


Рис. 1: 74HC595, накреслений на дошці: ліворуч п'ять керувальних виводів (SER, SRCLK, RCLK, SRCLR, OE), праворуч вісім виходів Q_A–Q_H з виводом каскаду Q'_H до другого 595; унизу часова діаграма SER/SRCLK/RCLK для прикладу послідовності `01100101`, а праворуч три варіанти позначення (HC/LS/HTC), які трапилися під час пошуку компонента.

2 П'ять керувальних виводів

Визначення

74HC595 має **п'ять виводів**, призначених для **керування даними**:

- **SER**: **послідовний вхід даних**, з'єднаний прямо з входом D **першого тригера** всередині. Це провід, на який **по одному біту** подають послідовність 0 і 1, яку треба завантажити.
- **SRCLK**: **clock регістра зсуву**. Кожен **передній фронт просуває на одну позицію** біт, щойно поданий на SER – точно як clock D-тригера.
- **RCLK** (*Register Clock*): **виводить на виходи** всі разом 8 бітів, завантажених у **внутрішній регістр зсуву**. Він не змінюється під час завантаження окремих бітів і отримує **один передній фронт** лише наприкінці, коли всі 8 бітів уже завантажено.
- **SRCLR** (*Serial Clear*, інверсний): якщо подати **низький рівень**, він **скидає всі виходи**. Його треба завжди тримати **під'єднаним** до $+5$ В (високий рівень, тобто "не активний"), бо ми не хочемо, щоб регістр **колись скидався** під час звичайної роботи.
- **OE** (*Output Enable*, інверсний): **вмикає виходи**. Його треба завжди **під'єднувати до GND**

(низький рівень, тобто "**завжди** **ввімкнено**").

У нашому використанні SRCLR і OE **незмінні** (відповідно на $+5$ В і GND): єдині **три** **виводи**, якими мікроконтролер справді має **керувати**, — це **SER**, **SRCLK** і **RCLK**.

Примітка

Вивід **OE** керує властивістю виходів, яка називається **tristate** (або *three-state*): крім звичайних **рівнів** **0** **і** **1**, вихід tristate може мати третій стан — **високий** **імпеданс**, коли він **фізично** **від'єднаний** від решти схеми. Ця функція потрібна, наприклад, коли кілька пристроїв мають **використовувати** **один** **провід** (шину): **лише** **один** **одночасно** "говорить" у провід, а всі інші залишаються від'єднаними завдяки tristate. У нашому використанні ми **не** **використовуємо** цю функцію, бо тримаємо OE завжди на **низькому** **рівні** (виходи завжди ввімкнені).

Приклад

Щоб навести конкретний приклад, на уроці обрали довільну **послідовність** **з** **8** **бітів**: `0 1 1 0 0 0 1 0`. З цією послідовністю **процедура** запису **байта** на **8** **виходів** 595 така:

1. **Подають** **перший** **біт** (`0`) на SER.
2. Створюють передній фронт на SRCLK: біт **просувається** у внутрішньому регістрі зсуву.
3. **Повторюють** для кожного з **8** **бітів** послідовності, по порядку: `0, 1, 1, 0, 0, 0, 1, 0`.
4. Лише після того, як завантажено **усі** **8** **бітів**, створюють один передній фронт на **RCLK**: тепер, і **лише** **тепер**, дані **виводяться** **на** **виходи**.

Примітка

Чому біт треба подавати **посередині** **проміжку** **часу**, відведеного на цей біт, а не одразу на початку? Бо на початку сигнал ще надто близько до **переходу** попереднього біта, і є ризик прочитати значення, яке **ще** **змінюється**. Це як стрибати на сходинку: стрибають не на край, а **на** **середину**, щоб точно влучити.

Приклад

Скільки на практиці має тривати кожен біт? Це залежить від того, наскільки **швидко** треба "заповнити" регістр. **Панель** **із** **8192** **світлодіодів** (1024 байти), яку треба оновлювати за секунду, потребує значно вищої **частоти** **SRCLK**, ніж один 595: щоб зображення здавалося плавним для **людського** **ока**, треба **оновлювати** **картинку** щонайменше з частотою **20 Гц**, а краще **50 Гц** – це означає працювати з часом на біт порядку **мікросекунд** **або** **мілісекунд**, а не секунд.

Увага

Навіщо потрібні **два** окремі регістри (**внутрішній регістр зсуву**, яким керує **SRCLK**, і **вихідний регістр**, яким керує **RCLK**), а не один? Якби виходи оновлювалися на **кожному такті** SRCLK, на 8 під'єднаних світлодіодах було б видно **зсув** даних біт за бітом, а не **миттєву зміну**. Це як **накрити трубку** з кульками, поки її заповнюють, і відкрити її (зробити фотографію) лише тоді, коли всі кульки на місці: **розділивши два clock**, зміна на виходах виглядає **миттєвою і чистою**, без проміжного етапу.

3 Вісім виходів і каскадне з'єднання

Визначення

Вісім виходів 74HC595 називаються $Q_A, Q_B, Q_C, Q_D, Q_E, Q_F, Q_G, Q_H$. До кожного можна під'єднати **будь-яке навантаження** (**світлодіод** з його **обмежувальним резистором**, транзистор тощо): примітно те, що для керування **8 незалежними виходами** займають лише **три** проводи мікроконтролера (SER, SRCLK, RCLK).

Приклад

Є дев'ятий провід, який називається Q'_H (Q_H зі штрихом): це **копія сигналу**, який усередині виходив би з дев'ятого тригера, якби ланцюжок продовжувався. Якщо з'єднати Q'_H першого 595 з виводом SER **другого 595** (і **зробити спільними SRCLK і RCLK** для всіх мікросхем у каскаді), отримаємо **16 виходів**, якими так само керують лише три проводи мікроконтролера. У **каскад** можна поставити скільки завгодно 595, теоретично без **фізичної межі**: **одноколірна світлодіодна панель** 32×16 модулів (усього 512 світлодіодів) працює саме за цим принципом, і **рекламні щити** чи великі екрани на концертах побудовані за тією самою логікою, лише в значно більшому масштабі.

4 Креслимо схему в KiCad

Примітка

Коли шукаєш компонент у **бібліотеці KiCad**, з'являється кілька варіантів (`74HC595`, `74HCT595`, `74LS595` тощо): вибір логічного компонента **не критичний**, **будь-який варіант** підходить для схеми. Але варто одразу задати правильний **футпринт** (*impronta*), бо зі зміною параметрів пошуку змінюються і доступні футпринти – краще вибрати його **один раз**, ніж потім шукати знову.

4.1 Панелька (socket)

Визначення

Панелька (*zoccolo*, *socket*) — це **пластикова основа** з виводами, які **припаюють до плати**, і з гніздами зверху, у які вставляють сам компонент, **не припаюючи** його напряму. Вона дозволяє **знімати і ставити** компонент **без відпаювання**: це корисно, якщо помітиш, що поставив його **навпаки** (досить вийняти і повернути), або якщо він згорить (виймаєш і ставиш новий без повторного паяння).

Приклад

Це не теоретичний ризик: трапляється, що компонент ставлять на плату навпаки, не перевіривши **позначку-ключ** (*tacca*) – якби компонент було припаяно напряму, **без панельки**, його довелося б **відпаювати** і ставити заново, а це довга й делікатна робота (**реальний ризик** пошкодити **доріжки** друкованої плати під час цього).

Приклад

Мікросхема може коштувати й **кілька євро** (наприклад, 15 €): якщо схему доводиться викинути, бо щось не працює, з панелькою втрачаєш **лише саму панельку** (кілька центів), а не мікросхему. Для тих, хто ще вчиться – коли ймовірність **помилки монтажу** ніколи не дорівнює нулю – ставити **завжди панельку обов'язково**, а не за бажанням.

Примітка

Існують **два типи конструкції** панельок: **пластинчасті** (*a lamella*), де зігнутий **металевий язичок** утворює "заходження", у яке вставляють вивід компонента, і він затискається між двома пластинками; і **цангові** (*a tulipano*), де контакт — маленький **суцільний циліндр** (точений) з отвором посередині, у який входить вивід – **міцніші й точніші**, але й **дорожчі**. Для **шкільного використання** майже завжди беруть пластинчасті, хоча вони трохи делікатніші й можуть згинатися з часом.

Примітка

Існує й третя категорія — панелька **ZIF** (*Zero Insertion Force*): вона працює за тим самим принципом монтажу, що й гнізда процесорів у настільних комп'ютерах – **важіль** піднімають, щоб вставити компонент **без зусилля**, потім опускають, щоб **зафіксувати** його. Для наших схем її **не використовують**: вона призначена для значно делікатніших і дорожчих компонентів, ніж 74HC595.

4.2 Вибір правильних футпринтів

Визначення

Кожному компоненту, крім електричного символу, треба призначити **футпринт** (*impronta*), який описує **фізичне креслення** на PCB:

- **Резистор**: тип `THT` (**наскрізний отвір**, не SMD), з кроком `P10,16` (10,16 мм між двома виводами).
- **Світлодіод**: **5 мм**, версія **без прозорого купола** (*flat top*).
- **Роз'єм до мікроконтролера**: тип `pin header`, **стандартний крок** 2,54 мм, **вертикальна** орієнтація (щоб відповідати типовим **роз'ємам розширення** плати Arduino).
- **595**: корпус `DIP`, з **socket** (панелькою) замість прямого паяння.

Примітка

Щоб **призначити футпринт**: **виділити компонент**, натиснути клавішу `F` (footprint), відкрити бібліотеку відповідною іконкою і скористатися **двома фільтрами** вгорі ("фільтр за виводом", "фільтр за кількістю pin"), щоб **швидко звузити пошук** перед вибором.

Увага

Вибираючи футпринт панельки між виводами **long pad** (видовженими) і **round pad** (круглими), для **ручного паяння** тими, хто ще вчиться, завжди краще брати **long pad**: **менша площадка** швидше нагрівається під час паяння, і є ризик, що **клей**, який тримає мідь на друкованій платі, розплавиться і **площадка відірветься**. **Більша площадка** потребує трохи більше припою, але дає **більше часу**, щоб завершити паяння без ризику відриву, і вона також **механічно міцніша**.

Приклад

Типова помилка: вибрати **неправильний роз'єм** для J1 (роз'єму до мікроконтролера), не дотримавшись уже даної вказівки. **Процедура виправлення**, крок за кроком: на електричній схемі виділити роз'єм, натиснути `F`, щоб відкрити бібліотеку, і поставити позначки у **двох прапорцях** "застосувати фільтри футпринтів" – один для фільтра за **кількістю виводів**, другий за **кроком**. Прокрутивши список до правильного кроку (2,54 мм, вертикальний), вибирають **правильний футпринт** та імпортують його. Після зміни футпринта треба зайти в панель "ports", щоб **перевірити й перемістити** всі вже зроблені **з'єднання**, бо зі зміною футпринта може змінитися **фізичне розташування виводів**, які треба з'єднати.

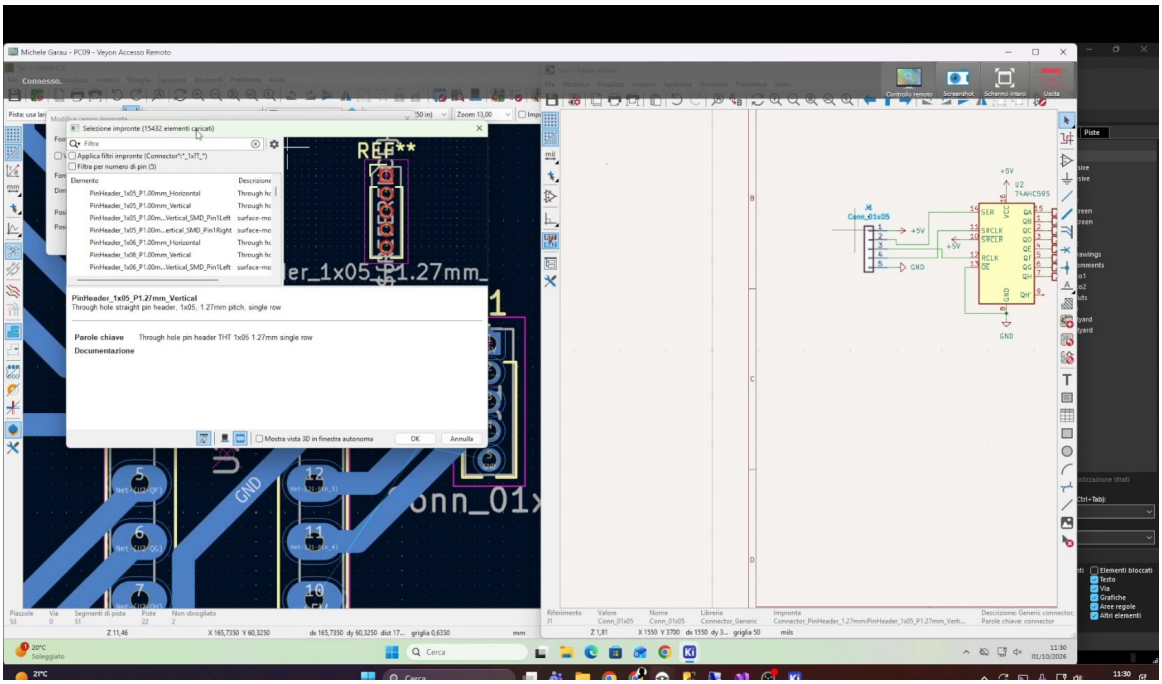


Рис. 2: Вікно “Вибір футпринтів” у KiCad під час пошуку футпринта роз’єму J1: початковий список показує кілька варіантів `PinHeader_1x05` з іншим кроком, ніж нам потрібно (1,00 мм, 1,27 мм) – треба прокрутити список до вертикального варіанта з кроком **2,54 мм**, який відповідає роз’ємам розширення плати Arduino. Праворуч — електрична схема 595 з J1, уже під’єднаним до п’яти сигналів.

4.3 Під’єднання живлення і розташування компонентів

Приклад

Живлення і **землю** (*massa*) під’єднують стандартними мітками **+5V** і **GND**, взятими з вибору **символів живлення**. **Важливе правило**: коли вибрано **символ GND**, на всій схемі треба використовувати **завжди той самий** – **різні символи** для того самого рівня KiCad вважає електрично **роз’єднаними** проводами, навіть якщо на вигляд вони однакові.

Примітка

Щоб не під’єднувати окремий символ GND до кожного з 8 світлодіодів, краще накреслити **один провід**, який з’єднує їх усіх разом, з **одним символом GND** у кінці.

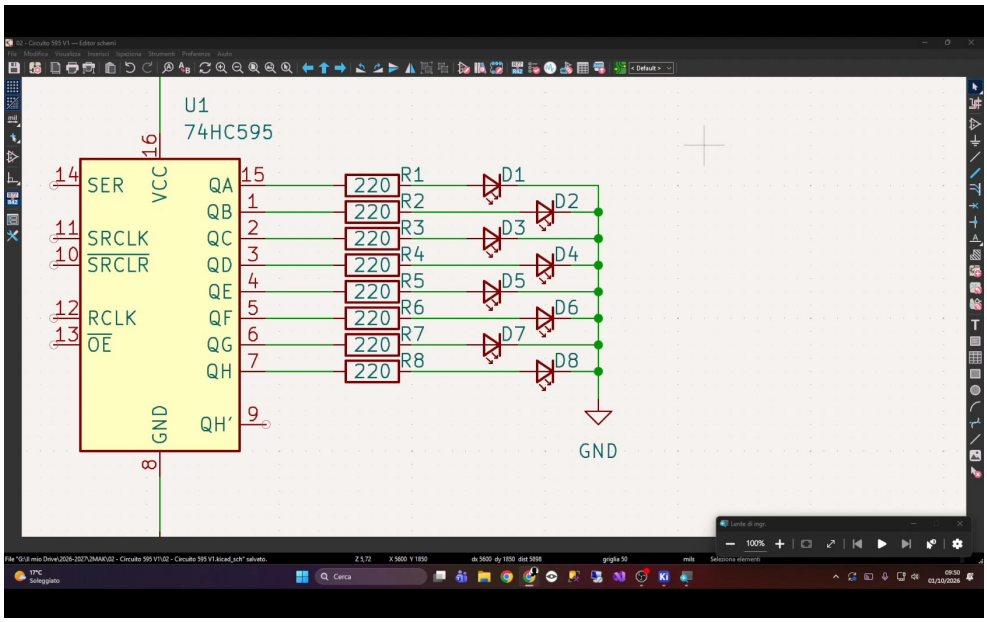


Рис. 3: Повна електрична схема восьми світлодіодів з відповідними резисторами 220 Ω, під'єднаних до виходів Q_A–Q_H мікросхеми 74HC595 (U1): один спільний провід землі з'єднує всі катоди з одним символом GND, замість повторювати його вісім разів.

Приклад

Щоб **скопіювати** **компонент**, який уже налаштовано (наприклад, резистор), на всі інші так, щоб мітки візуально не накладалися: спочатку **повністю** **доводять** один компонент (значення задано, мітки перенесено в охайне місце, футпринт призначено), і **лише** **потім** копіюють його на інші. **Спочатку** **копіювати, а** **потім** **поправляти** — це зайвий **графічний** **безлад**, який доводиться виправляти компонент за компонентом.

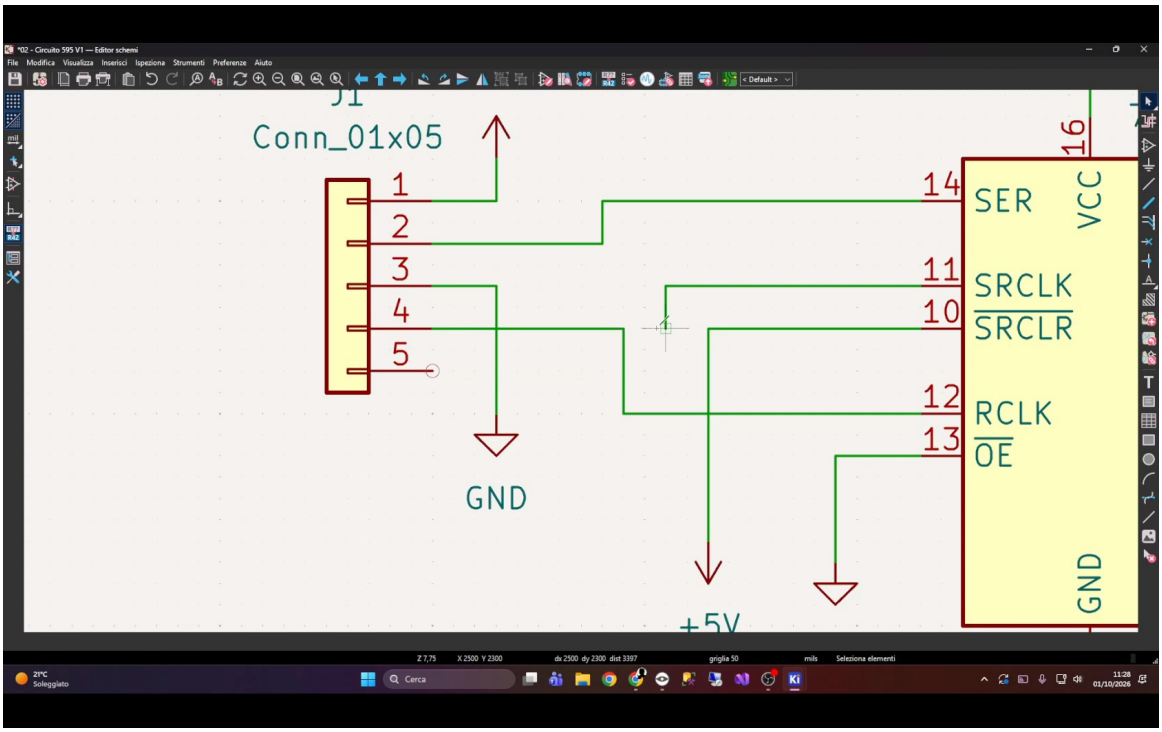


Рис. 4: Роз'єм J1 (до мікроконтролера/Arduino), під'єднаний до п'яти сигналів 595: вивід 1 — до SER, 2 — до SRCLK, 3 — до GND, 4 — до SRCLR (закріплений на +5 В проводом, що спускається від pin 4), 5 залишено вільним; RCLK і OE закріплені відповідно на виводі 12 і на GND.

Примітка

Рекомендоване естетичне розташування: тримати всі 8 світлодіодів **вирівняними** й **по порядку** (від Q_A до Q_H), а не розкиданими по платі – так значно легше з першого погляду прочитати **двійкові дані**. Цей вибір пов'язаний із **призначенням схеми**: якби ті самі світлодіоди мали бути частиною **панелі з фіксованими позиціями** через іншу вимогу (наприклад, написи, які треба підсвічувати), розташування підпорядковувалося б цій вимозі, а не двійковому порядку.

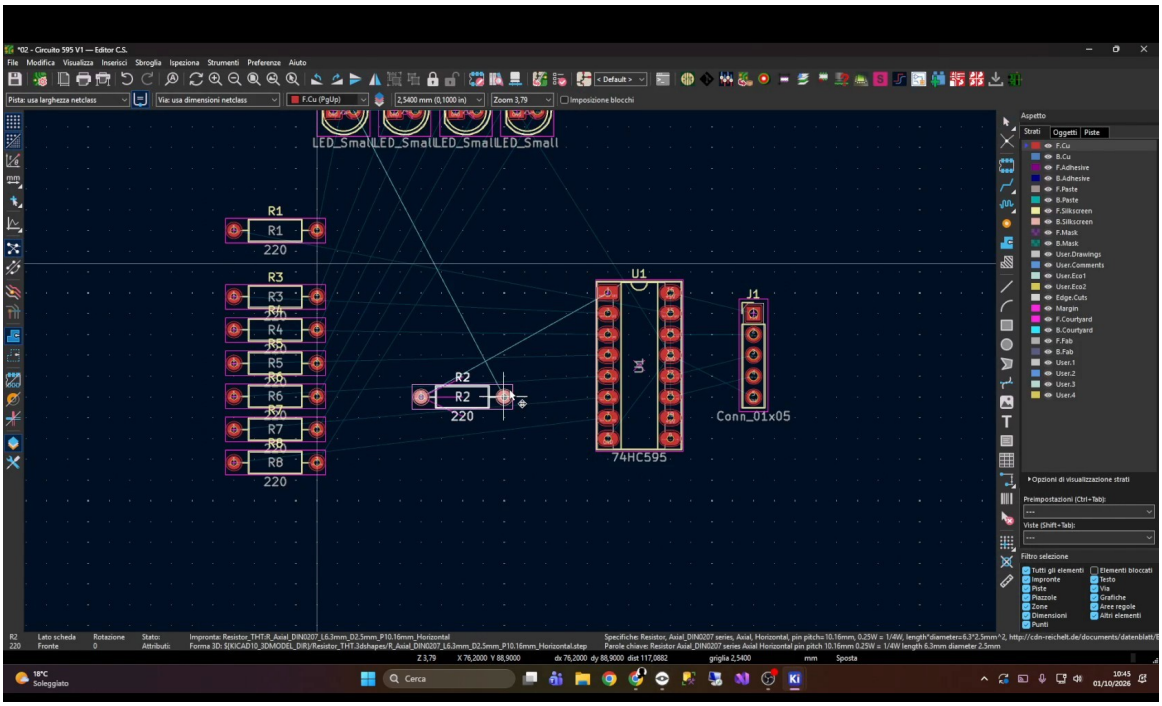


Рис. 5: Друкована плата на етапі розміщення: резистори (R1–R8) уже вирівняні вертикально ліворуч, 595 (U1) і роз'єм J1 (Conn_01x05) праворуч, а лінії ratsnest (тонкі, білі) показують, що справжні доріжки ще треба прокласти.

4.4 Ширина доріжок і правила розводки

Визначення

Net (мережа) — це сукупність точок схеми, електрично **еквіпотенціальних** (з'єднаних між собою тим самим проводом/доріжкою): кожна net має свій **ідентифікатор**. **Net class** — це налаштування (у KiCad: зелена іконка з шестернею → налаштування плати → net class), яке дозволяє задати **ширину** доріжок для **груп net** або для всіх разом (**клас за замовчуванням**).

Примітка

Мінімальна технологічна межа нашого процесу (**фотолітографії** вручну) — приблизно **0,4 мм**, і досягти її можна лише з **великою точністю**. Для такої простої схеми, як ця, без потреби в **надзвичайній щільності**, краще починати з **широких доріжок 1 мм**: значно більше за технічний мінімум, але **набагато толерантніше** до дрібних **неточностей виробництва**.

Увага

Тверде правило розводки: **ніколи не прокладати** доріжки з **поворотами під 90 градусів**. Правильно — **трохи відійти від площадки** і потім повернути, щоб завжди зберігати **безпечну відстань** між сусідніми доріжками – якщо дві доріжки залишаються надто близько, на етапі виробництва (ручне вирізання плати з заготовки) є ризик, що доведеться **зішкрібати зайву мідь канцелярським ножем**, з небезпечно малим **запасом на помилку**.

Приклад

Дві типові помилки, які виправляли під час розводки в класі:

- **Дві доріжки** йдуть **надто близько** одна до одної прямо від двох сусідніх площадок. Виправлення: коли починаєш від двох сусідніх площадок, **ніколи не прокладають** доріжки прямо поруч одна з одною від самого початку – **трохи відходять** від кожної площадки і лише потім повертають, залишаючи вільною найвужчу зону.
- Біля роз'єму J1 і проводу GND утворюється низка **ламаних поворотів** (провід GND спускався, ще спускався, а потім піднімався), які даремно подовжують шлях. Виправлення: вирівняти **вивід 3** роз'єму J1 з відповідним виводом GND — це дозволяє зробити **пряму доріжку** і повністю прибрати зайві повороти.

Щоб **виправити ділянку доріжки**, яку вже прокладено, не переробляючи все з нуля: виділити ділянку і переміщати її **клавішами зі стрілками** на клавіатурі. Можна також переміщати цілий компонент клавішею **G** (яка **тягне за собою** і вже зроблені з'єднання), але з обережністю: це не завжди працює правильно, і в деяких випадках може **заплутати з'єднання**, а не впорядкувати їх.

Увага

Робоча сітка: ніколи не використовуйте **сітку в цілих міліметрах** (наприклад, 5 мм), коли переміщаєте компоненти зі стандартним кроком 2,54 мм або 1,27 мм – компонент опиниться трохи навскіс відносно сітки виводів. Завжди використовуйте сітку **2,54 мм або 1,27 мм** (mils, одиниця вимірювання електронних компонентів; на міліметри переходять лише для механічних розмірів плати). Крім того, використовуйте **найбільшу можливу** сітку, яку дозволяє крок компонентів: **надто дрібна сітка** дозволяє непомітно зсунути компонент на мізерну відстань, а з **великою сіткою помилку розміщення** видно одразу. Це було, з чималим відривом, **правило, яке найчастіше повторювали** під час практичної роботи в класі – знак того, що його варто **завжди перевіряти двічі**, перш ніж переміщати компонент.

Приклад

Практичний критерій, щоб **мінімізувати довжину доріжок**: якщо компонент (наприклад, мікросхема U1) стоїть далі, ніж потрібно, від точки, з якою має з'єднуватися, його треба **пересунути ближче**. Якщо **вирівняти виводи** двох безпосередньо з'єднаних компонентів (наприклад, вивід роз'єму J1 з відповідним виводом GND), можна зробити **прямі доріжки** замість **ламаних поворотів**, які даремно подовжують шлях.

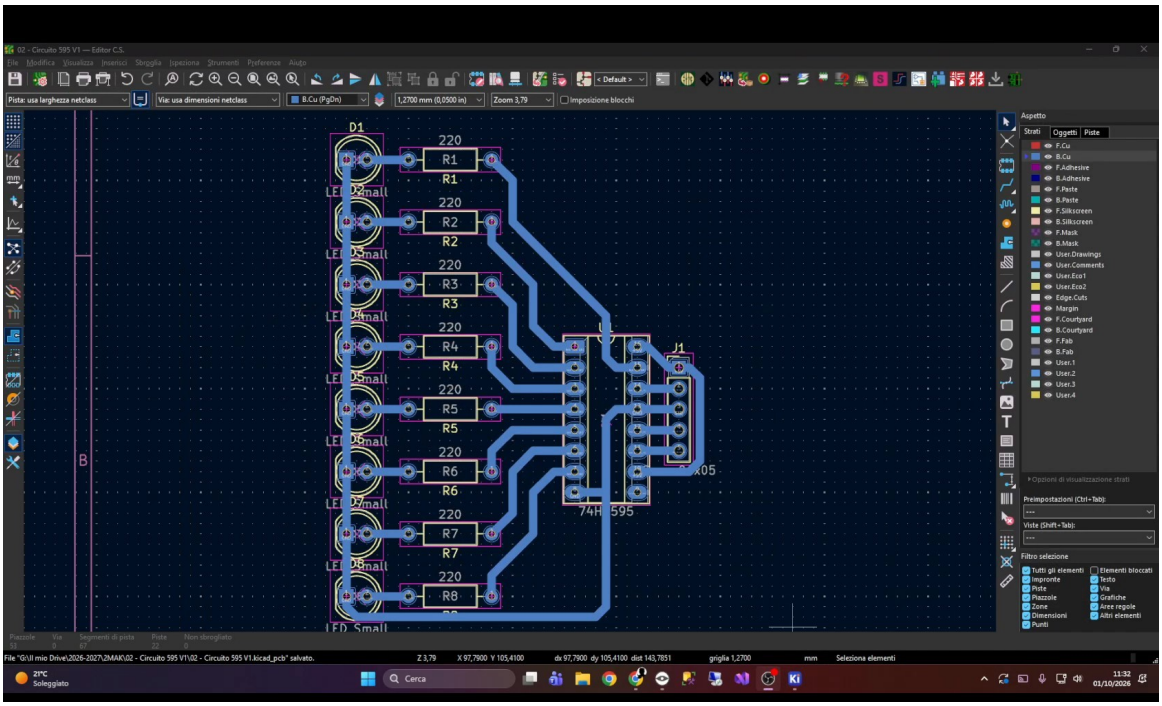


Рис. 6: Схема, розведена на 100% (“Non sbrogliato: 0”, унизу ліворуч): усі доріжки між резисторами, світлодіодами, 595 (U1) і роз’ємом J1 прокладено без кутів 90 градусів, з вісьмома світлодіодами, вирівняними вертикально ліворуч – це остаточний результат, якого має досягти розводка.

5 Від PCB до 3D-моделі

Визначення

Щоб задати **фізичні межі** плати, на шарі **Edge Cuts** треба накреслити прямокутник (або будь-яку іншу форму): без цього кроку **експорт 3D-моделі** не вдається, бо KiCad не знає, де “**лінія різу**” плати.

Примітка

Щоб **експортувати 3D-модель** плати (меню File → Esporta → STEP): **завжди обирайте** формат **STEP**, а не **STL**. STL — це **одне** тіло з трикутників, одноколірне: після імпорту в програму моделювання (наприклад, **Fusion 360**) окремі компоненти вже не можна **розділити чи сховати**. STEP, навпаки, експортує кожен компонент як **окрему** деталь (PCB, кожна мікросхема, кожен світлодіод тощо): у Fusion можна **переміщати, ховати чи моделювати** навколо кожної деталі окремо. Для такої схеми, як ця, імпорт STEP у Fusion дає приблизно **40 окремих деталей** (PCB, кожен резистор, кожен світлодіод, мікросхема з її панелькою, роз’єм тощо): ця кількість ще краще показує, чому **жорстко згрупувати їх** — це **перший крок**.

Увага

Після **імпорту STEP** у Fusion 360 **перший крок** — створити **групу жорсткого зв’язку** між **усіма компонентами**: без цього, переміщаючи плату, можна **зсунути окремий компонент** (напри-

клад, світлодіод), відокремивши його від решти й утративши **відповідність** **оригінальному** **кресленню**.

Примітка

Щоб **закріпити** **плату** всередині **коробки**, використовують **mounting** **hole** (**точки** **кріплення**): спеціальні компоненти, які, розміщені на схемі, задають, де робити **кріпильні** **отвори**. Поширена система кріплення — **на** **клацанні**: коробка має **гнучкий** **край**, який трохи відгинається, щоб вставити плату, а потім повертається на місце і фіксує її; щоб вийняти, досить **підділи** у правильному місці. Mounting hole **не** **завжди** **обов'язкові**: інколи кріпильних отворів можна взагалі уникнути, залежно від того, як плату потім насправді розміщують.

Приклад

Те, що обговорили в класі наперед і розглянемо на одному з наступних уроків: замість **окремого** **роз'єму** до Arduino можна вставити **символ** **самого** **Arduino** прямо в схему KiCad і спроєктувати **shield** – плату, яка вставляється прямо зверху на Arduino, **без** **навісних** **проводів**. Практична складність буде з паянням: у shield компоненти стоять зверху плати, а доріжки знизу, тому роз'єм, який вставляється в Arduino, треба паяти з протилежного боку від компонентів – ще одне обмеження монтажу порівняно зі схемою з окремим роз'ємом, розглянутою на цьому уроці.

6 Що запам'ятати з цього уроку

- **74HC595** керується лише трьома дійсними сигналами: **SER** (послідовні дані), **SRCLK** (просуває біт у внутрішньому регістрі зсуву), **RCLK** (виводить усі 8 бітів разом на виходи, без видимого зсуву).
- **SRCLR** і **OE** у нашому використанні завжди незмінні (відповідно на $+5$ В і GND).
- Кілька 595 можна з'єднати **каскадом** через вивід Q'_H, зі спільними SRCLK і RCLK: так **множаться** **виходи**, якими можна керувати, а на мікроконтролері залишається все одно **3** **проводи**.
- У KiCad: завжди призначати кожному компоненту правильний **футпринт** (резистор THT P10,16, світлодіод 5 мм flat top, вертикальний pin header 2,54 мм, 595 DIP з **socket**).
- **Ніколи** не робити поворотів під 90 градусів під час розводки доріжок; завжди використовувати сітку 2,54 мм або 1,27 мм, ніколи в цілих міліметрах, щоб залишатися вирівняними з кроком компонентів.
- Для 3D-моделі: завжди формат **STEP** (окремі компоненти), ніколи STL (одне тіло); завжди задавати шар **Edge** **Cuts** перед експортом.

Глосарій: італійські слова, які вживають на уроці

Italiano	**Українською**
uscita, ingresso	вихід, вхід
fronte di salita	передній фронт
pubblicare (sulle uscite)	вивести (на виходи)
collegamento in cascata	каскадне з’єднання
zoccolo	панелька (socket)
tacca di riferimento	позначка-ключ
saldare, dissaldare	паяти, відпаювати
stagno	припій
connettore	роз’єм
massa (GND)	земля (GND)
sbroglio	розводка
griglia	сітка
taglierino	канцелярський ніж
scatola, punti di fissaggio	коробка, точки кріплення